

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2**



Stack and Queue

Oleh:

Fathoni Akhmad Kurniawan

NIM. 2510817210013

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2: Struct and Point ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Fathoni Akhmad Kurniawan
NIM : 2510817210013

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S.Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN.....	ii
DAFTAR ISI	iii
DAFTAR GAMBAR.....	iv
DAFTAR TABEL	v
SOAL 1	1
A. Source code.....	1
B. Pembahasan	2
SOAL 2.....	5
A. Source code.....	6
B. Output Program	7
C. Pembahasan	7
SOAL 3.....	8
A. Source code.....	8
B. Pembahasan	9
SOAL 4.....	13
A. Source code.....	14
B. Output Program	15
C. Pembahasan	15
TAUTAN GIT	17

DAFTAR GAMBAR

Gambar 1 Screenshot Output Prob1.Cpp	7
Gambar 2 Screenshot Output Prob2.Cpp	15

DAFTAR TABEL

Table 1 Source Code Stack.Cpp	1
Table 2 Source Code Prob1.Cpp	6
Table 3 Source Code Queue.Cpp	8
Table 4 Source Code Queue.Cpp	14

SOAL 1

Pada file `stack.h` yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file `stack.cpp`. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan `throw`.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source code

Table 1 Source Code Stack.Cpp

1	<code>#include "stack.h"</code>
2	<code>#include <stdexcept></code>
3	
4	<code>void init(Stack* s) {</code>
5	<code> s->top = s->data - 1;</code>
6	<code>}</code>
7	
8	<code>bool isEmpty(const Stack* s) {</code>
9	<code> return s->top < s->data;</code>
10	<code>}</code>
11	
12	<code>bool isFull(const Stack* s) {</code>
13	<code> return s->top == s->data + MAX - 1;</code>
14	<code>}</code>
15	
16	<code>void push(Stack* s, int value) {</code>
17	<code> if (isFull(s)) {</code>
18	<code> throw std::overflow_error("Stack overflow: stack</code>
	<code>is full");</code>
19	<code> }</code>
20	<code> s->top++;</code>
21	<code> *(s->top) = value;</code>
22	<code>}</code>
23	
24	<code>void pop(Stack* s) {</code>
25	<code> if (isEmpty(s)) {</code>
26	<code> throw std::underflow_error("Stack underflow: stack</code>
	<code>is empty");</code>
27	<code>}</code>

```

28     s->top--;
29 }
30
31 int peek(const Stack* s) {
32     if (isEmpty(s)) {
33         throw std::underflow_error("Stack is empty: no
element to peek");
34     }
35     return *(s->top);
36 }

```

B. Pembahasan

Program pada Soal 1 terdiri atas dua berkas kode, yaitu `stack.h` dan `stack.cpp`. Kedua berkas tersebut bekerja secara terintegrasi untuk mengimplementasikan struktur data tumpukan (stack) berbasis larik statis dengan mekanisme pointer internal.

Berkas `stack.h` mendefinisikan struktur data `Stack` yang memuat dua anggota, yaitu sebuah larik bertipe `int` berukuran `MAX` (bernilai 100) sebagai wadah penyimpanan elemen, serta sebuah pointer bertipe `int*` yang diberi nama `top` untuk menunjuk elemen teratas tumpukan. Penggunaan `include guard` berupa direktif `#ifndef STACK_H`, `#define STACK_H`, dan `#endif` bertujuan untuk mencegah terjadinya deklarasi ganda apabila berkas header tersebut disertakan lebih dari satu kali dalam unit kompilasi yang sama.

Berkas `stack.cpp` menyertakan berkas `stack.h` dan mengimplementasikan seluruh enam fungsi yang dideklarasikan di dalamnya. Untuk menangani kondisi-kondisi tidak valid, digunakan mekanisme exception handling dengan melempar objek pengecualian bertipe `std::overflow_error` dan `std::underflow_error` dari pustaka standar `<stdexcept>`.

1. void init(Stack* s)

Algoritma: Pointer `top` diinisialisasi dengan nilai `s->data - 1`, yaitu satu posisi sebelum awal larik. Kondisi ini merepresentasikan tumpukan kosong karena `top` berada di luar batas larik yang valid.

Justifikasi: Dengan menggunakan konvensi ini, pemeriksaan `isEmpty` cukup dilakukan dengan membandingkan apakah `top` lebih kecil dari `s->data`, dan `isFull` dapat diperiksa dengan membandingkan `top` dengan elemen terakhir larik.

Kompleksitas Waktu: $O(1)$ — hanya satu operasi penugasan pointer.

Kompleksitas Ruang: $O(1)$ — tidak mengalokasikan memori tambahan.

2. `bool isEmpty(const Stack* s)`

Algoritma: Fungsi mengembalikan nilai true apabila $s \rightarrow \text{top} < s \rightarrow \text{data}$, yang berarti pointer top berada di bawah awal larik (kondisi kosong). Sebaliknya, mengembalikan false.

Justifikasi: Konvensi inisialisasi $s \rightarrow \text{data} - 1$ pada fungsi init menjamin bahwa kondisi ini hanya terpenuhi ketika belum ada elemen yang di-push atau seluruh elemen telah di-pop.

Kompleksitas Waktu: $O(1)$.

Kompleksitas Ruang: $O(1)$.

3. `bool isFull(const Stack* s)`

Algoritma: Fungsi mengembalikan nilai true apabila $s \rightarrow \text{top} == s \rightarrow \text{data} + \text{MAX} - 1$, yakni ketika pointer top menunjuk ke elemen terakhir larik. Kondisi ini menandakan tumpukan penuh.

Justifikasi: Karena larik berukuran MAX, indeks elemen terakhir yang valid adalah MAX - 1, sehingga alamat elemen terakhir adalah $s \rightarrow \text{data} + \text{MAX} - 1$.

Kompleksitas Waktu: $O(1)$.

Kompleksitas Ruang: $O(1)$.

4. `void push(Stack* s, int value)`

Algoritma: Pertama, dilakukan pemeriksaan apakah tumpukan penuh; apabila ya, dilempar pengecualian `std::overflow_error`. Jika tidak, pointer top dinaikkan satu posisi ($s \rightarrow \text{top}++$), kemudian nilai value disimpan ke lokasi yang ditunjuk oleh top.

Justifikasi: Kenaikan pointer dilakukan sebelum penyimpanan nilai (pre-increment implicit via $s \rightarrow \text{top}++$) sehingga elemen baru selalu ditempatkan pada posisi yang belum terisi. Urutan operasi ini menjamin konsistensi data.

Kompleksitas Waktu: $O(1)$.

Kompleksitas Ruang: $O(1)$ per pemanggilan; $O(n)$ secara akumulatif untuk n elemen.

5. void pop(Stack* s)

Algoritma: Pertama, dilakukan pemeriksaan apakah tumpukan kosong; apabila ya, dilempar pengecualian `std::underflow_error`. Jika tidak, pointer top diturunkan satu posisi (`s->top--`), sehingga elemen teratas tidak lagi dapat diakses melalui antarmuka Stack.

Justifikasi: Operasi pop tidak perlu menghapus data secara fisik dari memori karena data yang berada di luar jangkauan pointer top dianggap tidak valid oleh antarmuka. Penurunan pointer sudah cukup untuk merepresentasikan pengangkatan elemen.

Kompleksitas Waktu: $O(1)$.

Kompleksitas Ruang: $O(1)$.

6. int peek(const Stack* s)

Algoritma: Pertama, dilakukan pemeriksaan apakah tumpukan kosong; apabila ya, dilempar pengecualian `std::underflow_error`. Jika tidak, nilai yang ditunjuk oleh pointer top dikembalikan melalui dereferensi `*(s->top)`.

Justifikasi: Berbeda dengan pop, fungsi peek bersifat hanya baca (read-only), ditandai dengan parameter `const Stack*`. Oleh karena itu, pointer top tidak diubah. Fungsi ini berguna untuk mengintip nilai teratas tanpa mengubah kondisi tumpukan.

Kompleksitas Waktu: $O(1)$.

Kompleksitas Ruang: $O(1)$.

SOAL 2

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal,

mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

Setiap simbol bisa berupa:

- sebuah bilangan bulat, atau
- sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

- Mesin membaca simbol dari kiri ke kanan.
- Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.
- Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang

ditampilkan sebagai hasil akhir.

Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1000 \leq A_i \leq 1000$

Keterangan:

- n adalah jumlah simbol dalam ekspresi
- A_i adalah nilai bilangan pada input
- Simbol yang valid hanya:
 - bilangan bulat
 - operator: +, -, *, /

Format Input

n

$A_1 A_2 \dots A_n$

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang simbol
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Sebuah bilangan hasil dari kalkulator Si Palui.

Contoh kasus

3 2 3 +	5
9 5 1 2 + 4 * + 3 -	14
5 10 2 / 3 *	15

A. Source code

Table 2 Source Code Probl.Cpp

```

1  #include "stack.h"
2  #include <iostream>
3  #include <string>
4  #include <stdexcept>
5
6  int main() {
7      int n;
8      std::cin >> n;
9
10     Stack s;
11     init(&s);
12
13     for (int i = 0; i < n; i++) {
14         std::string token;
15         std::cin >> token;
16
17         if (token == "+" || token == "-" || token == "*"
18         || token == "/") {
19             int b = peek(&s); pop(&s);
20             int a = peek(&s); pop(&s);
21             int result;
22             if (token == "+") result = a + b;
23             else if (token == "-") result = a - b;
24             else if (token == "*") result = a * b;
25             else result = a / b;
26             push(&s, result);
27         } else {
28             push(&s, std::stoi(token));
29         }
30     }
31     std::cout << peek(&s) << std::endl;

```

```

32     return 0;
33 }

```

B. Output Program

```

PS C:\Users\denny\OneDrive\Documents\TUGAS KULIAH\semester 2\PRAKTIKUM ALGO\module-2-stack-and-queue-dennymauladinoor23-pdf> cd "C:\Users\denny\OneDrive\Documents\TUGAS KULIAH\semester 2\PRAKTIKUM ALGO\module-2-stack-and-queue-dennymauladinoor23-pdf"
>>
PS C:\Users\denny\OneDrive\Documents\TUGAS KULIAH\semester 2\PRAKTIKUM ALGO\module-2-stack-and-queue-dennymauladinoor23-pdf> g++ -std=c++17 stack.cpp prob1.cpp -o prob1.exe
>>
PS C:\Users\denny\OneDrive\Documents\TUGAS KULIAH\semester 2\PRAKTIKUM ALGO\module-2-stack-and-queue-dennymauladinoor23-pdf> .\prob1.exe
>>
3
2 3 +
5
PS C:\Users\denny\OneDrive\Documents\TUGAS KULIAH\semester 2\PRAKTIKUM ALGO\module-2-stack-and-queue-dennymauladinoor23-pdf> .\prob1.exe
>>
9
5 1 2 + 4 * + 3 -
14
PS C:\Users\denny\OneDrive\Documents\TUGAS KULIAH\semester 2\PRAKTIKUM ALGO\module-2-stack-and-queue-dennymauladinoor23-pdf> .\prob1.exe
>>
5
10 2 / 3 *
15
PS C:\Users\denny\OneDrive\Documents\TUGAS KULIAH\semester 2\PRAKTIKUM ALGO\module-2-stack-and-queue-dennymauladinoor23-pdf>

```

Gambar 1 Screenshot Output Prob1.Cpp

C. Pembahasan

Program pada Soal 2 terdiri atas dua berkas kode, yaitu `stack.h` beserta `stack.cpp` dan berkas program utama `prob1.cpp`. Program ini mengimplementasikan evaluasi ekspresi aritmetika dalam notasi postfix (Reverse Polish Notation / RPN), yaitu format yang digunakan oleh "mesin hitung aneh" dalam soal.

Berkas `prob1.cpp` menyertakan `stack.h` dan memanfaatkan struktur `Stack` beserta seluruh fungsinya untuk menyelesaikan permasalahan. Pada fungsi `main()`, pertama-tama dibaca nilai `n` sebagai jumlah simbol, kemudian sebuah objek `Stack` diinisialisasi menggunakan fungsi `init()`. Program kemudian membaca setiap token dari masukan satu per satu dalam sebuah perulangan sebanyak `n` iterasi.

Algoritma pemrosesan token berjalan sebagai berikut: apabila token yang dibaca adalah salah satu dari empat operator aritmetika (+, -, *, /), maka dua operand terbaru diambil dari tumpukan menggunakan fungsi `peek()` dan `pop()`. Operand pertama yang diambil (`b`) adalah operand kanan, dan operand kedua yang diambil (`a`) adalah operand kiri. Hal ini dikarenakan tumpukan bersifat LIFO (Last In First Out), sehingga operand yang baru terakhir dimasukkan harus diambil terlebih dahulu. Setelah operasi aritmetika dilakukan, hasilnya di-push kembali ke tumpukan. Apabila token bukan operator, maka token dikonversi menjadi bilangan bulat menggunakan `std::stoi()` dan langsung di-push ke tumpukan.

Setelah seluruh `n` token diproses, nilai akhir yang tersisa di puncak tumpukan merupakan hasil evaluasi ekspresi, yang kemudian dicetak menggunakan `peek()`.

SOAL 3

Pada file queue.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file queue.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis

kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source code

Table 3 Source Code Queue.Cpp

```
1  #include "queue.h"
2  #include <stdexcept>
3
4  void init(Queue* q) {
5      q->front = q->data;
6      q->rear  = q->data - 1;
7  }
8
9  bool isEmpty(const Queue* q) {
10     return q->rear < q->front;
11 }
12
13 bool isFull(const Queue* q) {
14     return (q->rear - q->front + 1) == MAX;
15 }
16
17 void enqueue(Queue* q, int value) {
18     if (isFull(q)) {
19         throw std::overflow_error("Queue overflow: queue
is full");
20     }
21
22     if (q->rear == q->data + MAX - 1) {
23         int current_size = q->rear - q->front + 1;
24         for (int i = 0; i < current_size; i++) {
25             q->data[i] = *(q->front + i);
26         }
27         q->front = q->data;
28         q->rear = q->data + current_size - 1;
29     }
```

```

30
31     q->rear++;
32     *(q->rear) = value;
33 }
34
35 void dequeue(Queue* q) {
36     if (isEmpty(q)) {
37         throw std::underflow_error("Queue underflow: queue
is empty");
38     }
39     q->front++;
40
41     if (isEmpty(q)) {
42         q->front = q->data;
43         q->rear = q->data - 1;
44     }
45 }
46
47 int front(const Queue* q) {
48     if (isEmpty(q)) {
49         throw std::underflow_error("Queue is empty: no
front element");
50     }
51     return *(q->front);
52 }
53
54 int back(const Queue* q) {
55     if (isEmpty(q)) {
56         throw std::underflow_error("Queue is empty: no
back element");
57     }
58     return *(q->rear);
59 }

```

B. Pembahasan

Program pada Soal 3 terdiri atas dua berkas kode, yaitu queue.h dan queue.cpp. Kedua berkas tersebut bekerja secara terintegrasi untuk mengimplementasikan struktur data antrian (queue) berbasis larik statis dengan mekanisme dua pointer, yaitu front dan rear.

Berkas queue.h mendefinisikan struktur data Queue yang memuat tiga anggota, yaitu sebuah larik bertipe int berukuran MAX (bernilai 100) sebagai wadah penyimpanan elemen, pointer front yang menunjuk ke elemen terdepan antrian, serta pointer rear yang menunjuk

ke elemen paling belakang. Penggunaan include guard dengan pola yang sama dengan stack.h diterapkan pula di sini untuk mencegah deklarasi ganda.

Berkas queue.cpp mengimplementasikan seluruh tujuh fungsi yang dideklarasikan. Pendekatan yang digunakan adalah antrean linier (linear queue) berbasis pointer, di mana operasi enqueue selalu menambahkan elemen di belakang (rear) dan operasi dequeue selalu mengangkat elemen dari depan (front) dengan cara menggeser pointer front maju.

Penjelasan Fungsi

1. void init(Queue* q)

Algoritma: Pointer front diinisialisasi ke $q \rightarrow \text{data}$ (posisi awal larik), sedangkan pointer rear diinisialisasi ke $q \rightarrow \text{data} - 1$ (satu posisi sebelum awal larik), merepresentasikan kondisi antrean kosong.

Justifikasi: Kondisi kosong diwakili oleh $\text{rear} < \text{front}$, yang secara alami terpenuhi pada inisialisasi ini.

Kompleksitas Waktu: $O(1)$. Kompleksitas Ruang: $O(1)$.

2. bool isEmpty(const Queue* q)

Algoritma: Mengembalikan true apabila $q \rightarrow \text{rear} < q \rightarrow \text{front}$, yakni ketika pointer rear berada di sebelum pointer front — kondisi yang hanya terjadi saat antrean kosong.

Justifikasi: Setelah setiap operasi enqueue, rear selalu berada di posisi yang sama atau lebih tinggi dari front. Setelah dequeue menggeser front melampaui rear, kondisi kosong terpenuhi.

Kompleksitas Waktu: $O(1)$. Kompleksitas Ruang: $O(1)$.

3. bool isFull(const Queue* q)

Algoritma: Mengembalikan true apabila $q \rightarrow \text{rear} == q \rightarrow \text{data} + \text{MAX} - 1$, yaitu ketika pointer rear telah mencapai elemen terakhir larik.

Justifikasi: Implementasi ini menggunakan antrean linier, sehingga kapasitas larik habis ketika rear mencapai ujung akhir, meskipun secara fisik posisi awal larik (yang telah di-dequeue) mungkin masih kosong.

Kompleksitas Waktu: $O(1)$. Kompleksitas Ruang: $O(1)$.

4. void enqueue(Queue* q, int value)

Algoritma: Pertama diperiksa apakah antrean penuh; apabila ya, dilempar `std::overflow_error`. Jika tidak, pointer rear dinaikkan satu posisi, kemudian value disimpan pada lokasi yang ditunjuk rear.

Justifikasi: Penambahan elemen dilakukan dari sisi belakang antrean sesuai sifat FIFO (First In First Out). Kenaikan pointer rear sebelum penyimpanan memastikan elemen ditempatkan pada posisi berikutnya yang tersedia.

Kompleksitas Waktu: $O(1)$. Kompleksitas Ruang: $O(1)$ per pemanggilan.

5. void dequeue(Queue* q)

Algoritma: Pertama diperiksa apakah antrean kosong; apabila ya, dilempar `std::underflow_error`. Jika tidak, pointer front digeser maju satu posisi (`q->front++`), sehingga elemen terdepan tidak lagi dapat diakses melalui antarmuka Queue.

Justifikasi: Seperti halnya pop pada Stack, dequeue tidak perlu menghapus data secara fisik. Penggeseran pointer front ke depan sudah cukup untuk merepresentasikan pengangkatan elemen terdepan.

Kompleksitas Waktu: $O(1)$. Kompleksitas Ruang: $O(1)$.

6. int front(const Queue* q)

Algoritma: Setelah memeriksa kondisi kosong, fungsi mengembalikan nilai yang ditunjuk oleh pointer `q->front` melalui dereferensi `*(q->front)`.

Justifikasi: Fungsi ini bersifat read-only (parameter `const Queue*`) dan memberikan akses ke elemen terdepan tanpa mengubah kondisi antrean.

Kompleksitas Waktu: $O(1)$. Kompleksitas Ruang: $O(1)$.

7. int back(const Queue* q)

Algoritma: Setelah memeriksa kondisi kosong, fungsi mengembalikan nilai yang ditunjuk oleh pointer `q->rear` melalui dereferensi `*(q->rear)`.

Justifikasi: Simetris dengan fungsi front, fungsi ini memberikan akses ke elemen paling belakang antrian secara read-only.

Kompleksitas Waktu: $O(1)$. Kompleksitas Ruang: $O(1)$.

SOAL 4

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan.

Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1 \leq k \leq n$
- $-1000 \leq A_i \leq 1000$

Keterangan:

- n : jumlah elemen
- k : ukuran jendela
- A_i : elemen array

Format Input

n k

A_1 A_2 ... A_n

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang array. Lalu k besar atau jumlah elemen dalam satu kelompok yang dibentuk
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi

Contoh Kasus

5 3 1 2 3 4 5	6 9 13
5 5 1 2 3 4 5	15
6 2 4 -1 2 -3 5 1	3 1 -1 2 6

A. Source code

Table 4 Source Code Queue.Cpp

1	#include "queue.h"
2	#include <iostream>
3	
4	int main() {
5	int n, k;
6	std::cin >> n >> k;
7	
8	int arr[100];
9	for (int i = 0; i < n; i++) {
10	std::cin >> arr[i];
11	}
12	
13	Queue q;
14	init(&q);
15	
16	int windowSum = 0;
17	for (int i = 0; i < k; i++) {
18	enqueue(&q, arr[i]);
19	windowSum += arr[i];
20	}
21	
22	bool first = true;
23	for (int i = k; i <= n; i++) {
24	if (!first) std::cout << " ";
25	std::cout << windowSum;
26	first = false;
27	
28	if (i < n) {
29	windowSum -= front(&q);
30	dequeue(&q);
31	enqueue(&q, arr[i]);
32	windowSum += arr[i];

```

33         }
34     }
35     std::cout << std::endl;
36     return 0;
37 }

```

B. Output Program

```

PS C:\Users\denny\OneDrive\Documents\TUGAS KULIAH\semester 2\PRAKTIKUM ALGO\module-2-stack-and-queue-dennymauladinoor23-pdf> cd "C:\Users\denny\OneDrive\Documents\TUGAS KULIAH\semester 2\PRAKTIKUM ALGO\module-2-stack-and-queue-dennymauladinoor23-pdf"
>>
PS C:\Users\denny\OneDrive\Documents\TUGAS KULIAH\semester 2\PRAKTIKUM ALGO\module-2-stack-and-queue-dennymauladinoor23-pdf> g++ -std=c++17 queue.cpp prob2.cpp -o prob2.exe
>>
PS C:\Users\denny\OneDrive\Documents\TUGAS KULIAH\semester 2\PRAKTIKUM ALGO\module-2-stack-and-queue-dennymauladinoor23-pdf> .\prob2.exe
>>
5 3
1 2 3 4 5
6 9 12
PS C:\Users\denny\OneDrive\Documents\TUGAS KULIAH\semester 2\PRAKTIKUM ALGO\module-2-stack-and-queue-dennymauladinoor23-pdf> .\prob2.exe
>>
5 5
1 2 3 4 5
15
PS C:\Users\denny\OneDrive\Documents\TUGAS KULIAH\semester 2\PRAKTIKUM ALGO\module-2-stack-and-queue-dennymauladinoor23-pdf> .\prob2.exe
>>
6 2
4 -1 2 -3 5 1
3 1 -1 2 6
PS C:\Users\denny\OneDrive\Documents\TUGAS KULIAH\semester 2\PRAKTIKUM ALGO\module-2-stack-and-queue-dennymauladinoor23-pdf>

```

Gambar 2 Screenshot Output Prob2.Cpp

C. Pembahasan

Program pada Soal 4 terdiri atas dua berkas kode, yaitu queue.h beserta queue.cpp dan berkas program utama prob2.cpp. Program ini mengimplementasikan algoritma sliding window sum, yakni penghitungan jumlah elemen pada sebuah jendela (window) berukuran k yang digeser satu langkah ke kanan pada setiap iterasi sepanjang larik masukan.

Berkas prob2.cpp menyertakan queue.h dan memanfaatkan struktur Queue untuk memodelkan jendela geser secara efisien. Pada fungsi main(), pertama-tama dibaca nilai n (jumlah elemen) dan k (ukuran jendela), diikuti dengan pembacaan seluruh elemen larik arr. Sebuah objek Queue kemudian diinisialisasi menggunakan fungsi init().

Algoritma pengisian jendela awal dilakukan dengan men-enqueue k elemen pertama dari larik ke dalam antrean, sambil menghitung jumlah awal (windowSum). Setelah jendela awal terisi, program memasuki perulangan yang berjalan dari posisi k hingga n. Pada setiap iterasi, nilai windowSum dicetak sebagai hasil untuk jendela saat ini. Kemudian, apabila masih tersisa elemen berikutnya, nilai elemen terdepan antrean (yang keluar dari jendela) dikurangkan dari windowSum, di-dequeue dari antrean, dan elemen berikutnya dari larik (arr[i]) ditambahkan ke windowSum serta di-enqueue ke antrean.

Justifikasi Kebenaran Algoritma

Algoritma ini terbukti benar karena antrean secara tepat merepresentasikan jendela berukuran k yang sedang aktif. Sifat FIFO antrean menjamin bahwa elemen yang dimasukkan pertama (paling kiri dalam jendela) akan menjadi elemen yang dikeluarkan pertama kali ketika jendela digeser ke kanan. Dengan demikian, variabel `windowSum` selalu mencerminkan jumlah tepat k elemen dalam jendela aktif saat ini.

Pendekatan ini lebih efisien dibandingkan menghitung ulang jumlah setiap jendela dari nol, karena setiap pergeseran jendela hanya memerlukan satu operasi pengurangan dan satu operasi penjumlahan, bukan k operasi penjumlahan.

Analisis Kompleksitas

Kompleksitas Waktu: $O(n)$, di mana n adalah jumlah elemen larik. Setiap elemen di-enqueue tepat satu kali dan di-dequeue tepat satu kali, dan seluruh operasi antrean berjalan dalam $O(1)$. Jumlah total jendela yang diproses adalah $n - k + 1$, namun seluruh proses tetap $O(n)$.

Kompleksitas Ruang: $O(k)$, karena antrean hanya menyimpan elemen-elemen dalam jendela aktif berukuran k pada setiap saat.

TAUTAN GIT

<https://github.com/Fath-wan/ALGORITMA-DAN-STRUKTUR-DATA-MODUL-2.git>